

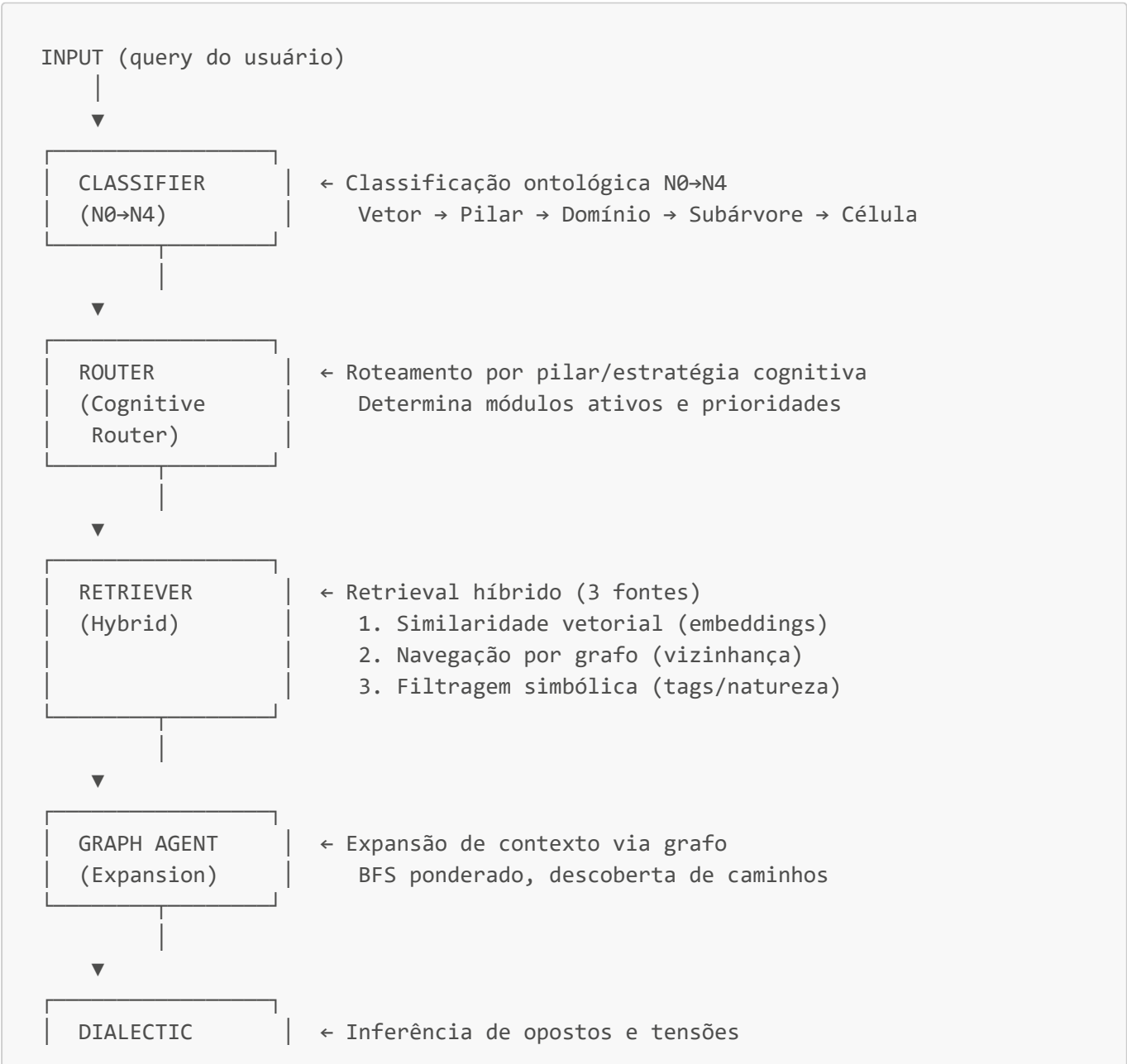
ARQUITETURA DO SISTEMA COGNITIVO KILO CODE

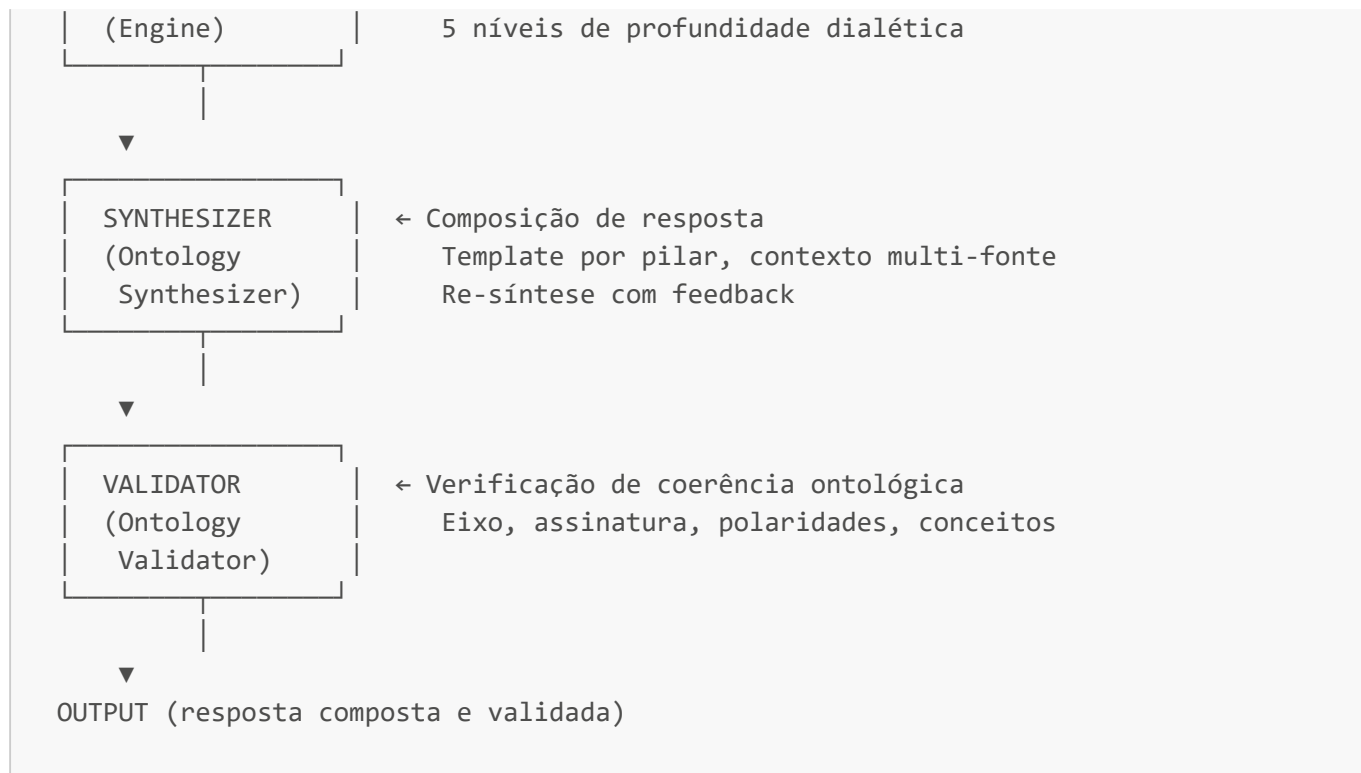
Versão: 1.0
Status: Operacional — Pipeline validado end-to-end
Base Ontológica: 162 células N4 (2×3×3×3×3)
Entrada Principal: `run_kilo.py`

1. VISÃO GERAL

O Kilo Code é um **sistema cognitivo ontológico operacional** que transforma uma ontologia fractal estática em um motor de inferência, recuperação e síntese. A arquitetura segue o princípio de separação de responsabilidades com degradação graciosa: cada módulo opera independentemente e falhas em uma camada não quebram o pipeline.

1.1 Pipeline Principal





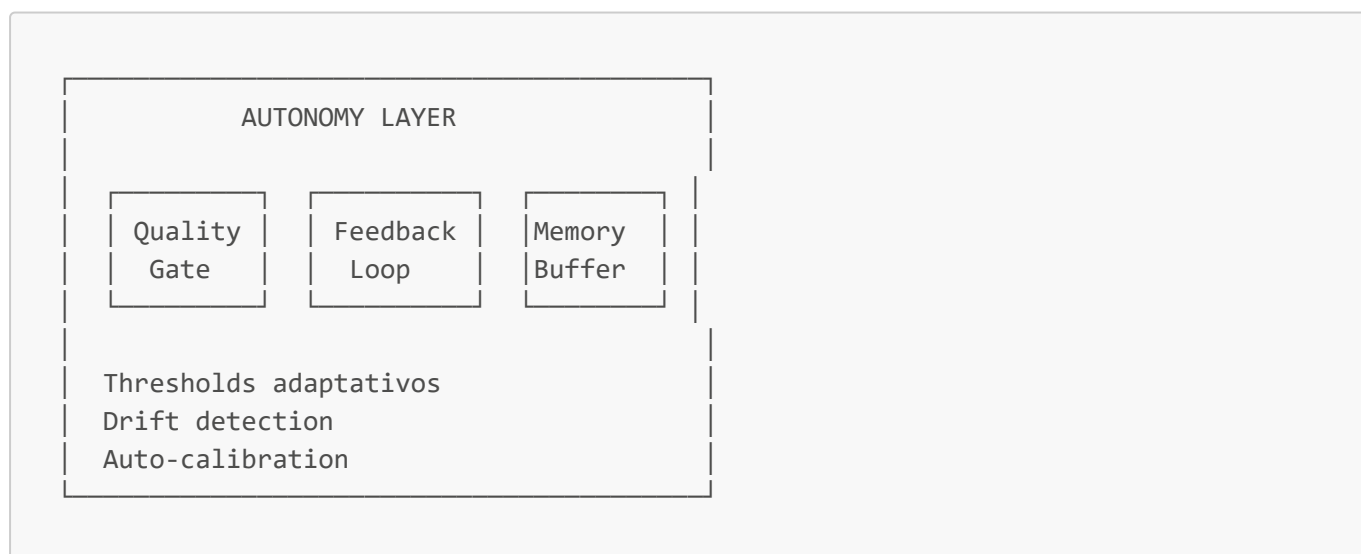
1.2 Pipeline Simplificado

Para queries de baixa complexidade ou quando recursos são limitados:

INPUT → ROUTER → RETRIEVER → SYNTHESIZER → VALIDATOR → OUTPUT

1.3 Autonomy Layer (Modo Autônomo)

Camada de auto-avaliação que envolve qualquer pipeline:



2. ESTRUTURA DE DIRETÓRIOS

```

e:/Arquivos/Área de Trabalho/MestreCuca/
├── run_kilo.py                                # Entry point principal (CLI)
├── core/                                       # Motores centrais (framework agnóstico)
│   ├── __init__.py
│   ├── agent_base.py                        # BaseAgent, AgentResult, AgentMemory
│   ├── ontology_graph.py                   # OntologyGraph (NetworkX)
│   ├── signature_engine.py                 # Assinaturas semânticas 9D
│   ├── dialectic_engine.py                 # Inferência de opostos (1018 linhas)
│   ├── cognitive_function_engine.py        # Funções cognitivas
│   ├── ontology_typing.py                  # Naturezas ontológicas
│   └── relation_engine.py                  # Relações tipadas
├── runtime/                                  # Módulos de execução do pipeline
│   ├── classifier.py                       # OntologicalClassifier (949 linhas)
│   ├── router.py                           # CognitiveRouter (450 linhas)
│   ├── retriever.py                        # HybridRetriever (475 linhas)
│   ├── synthesizer.py                      # OntologySynthesizer (647 linhas)
│   ├── validator.py                        # OntologyValidator (732 linhas)
│   └── orchestrator.py                     # MultiAgentOrchestrator (563 linhas)
├── .kilo/                                    # Configuração e agentes
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── classifier_agent.py             # Agente de classificação (360 linhas)
│   │   ├── graph_agent.py                  # Agente de grafo (454 linhas)
│   │   ├── synthesis_agent.py              # Agente de síntese (292 linhas)
│   │   ├── validator_agent.py              # Agente de validação (471 linhas)
│   │   ├── dialectic_agent.py              # Agente dialético (545 linhas)
│   │   ├── driver.py                       # Driver interativo (com issues)
│   │   ├── health_check.py                 # Health check (13 verificações)
│   │   ├── autonomy_layer.py               # Auto-avaliação (~550 linhas)
│   │   └── multi_agent_orchestrator.py     # Orquestrador (~480 linhas)
│   ├── config/
│   │   ├── ontology.yaml                   # Configuração ontológica
│   │   ├── embedding.yaml                  # Configuração de embeddings
│   │   ├── retrieval.yaml                  # Configuração de retrieval
│   │   └── graph.yaml                      # Configuração do grafo
│   ├── prompts/
│   │   ├── classifier/
│   │   ├── retrieval/
│   │   ├── routing/
│   │   ├── synthesis/
│   │   └── validation/
│   └── memory/
│       └── memory.yaml                     # Configuração de memória
├── data/                                     # Dados gerados e processados
│   ├── json/
│   │   └── embeddings_index.json           # 162 JSONs N4 enriquecidos
│   ├── embeddings/
│   │   └── embeddings_index.json           # 162 vetores .npy (384-dim)

```

```
|   └─ N4_*.npy (162 arquivos)
└─ graphs/
    │   └─ ontology.gexf          # Grafo exportado (891KB)
    │   └─ ontology.json         # Grafo em JSON (603KB)
    │   └─ graph_metrics.json    # Métricas topológicas (86KB)
└─ semantic_expansion_report.md
└─ test_results.csv
└─ test_results.json

└─ tools/                                # Ferramentas de build/geração
    │   └─ n4_to_json.py          # Conversor N4 → JSON enriquecido
    │   └─ uid_generator.py       # Gerador de UUIDs permanentes
    │   └─ embedding_builder.py   # Geração de embeddings
    │   └─ graph_builder.py       # Construção do grafo
    │   └─ canonical_document_builder.py # Docs canônicos
    │   └─ enrich_json.py         # Enriquecimento de JSONs
    │   └─ build_embeddings.py
    │   └─ validate_deep.py
    │   └─ validate_fase1.py
    │   └─ e2e_test.py
    │   └─ inspect_json.py
    │   └─ diag*.py              # Diagnósticos
└─ docs/                                # Documentação
    │   └─ roadmap/
    │       │   └─ 1 - Semantic Operating Substrate - roadmap.md
    │       │   └─ 1.1 - Roadmap-ImpN5.yaml
    │       │   └─ 2 - COGNITIVE_EXPRESSION_LAYER_ROADMAP.md
    │       │   └─ 3 - Semantic Cognitive Infrastructure Stack.md
    │   └─ arquitetura_sistema_cognitivo.md ← ESTE ARQUIVO

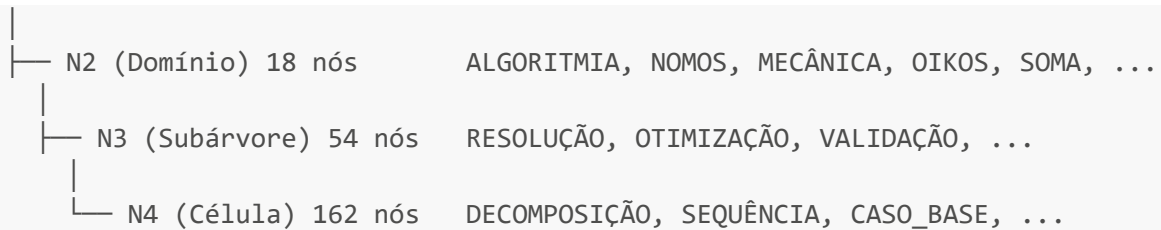
└─ ontology/                            # Fontes ontológicos
└─ prompts/                             # Templates de prompt
└─ scripts/                             # Scripts auxiliares
└─ tests/                               # Testes automatizados

└─ N3-*.md                             # 54 arquivos N3 (subárvores)
└─ N4-*.md                             # 144+ arquivos N4 (células)
└─ CORE.md                             # Especificação do core
└─ CORE2.md                            # Especificação core v2
└─ ARQUITETO_PROMPTS_FINAL.md          # Base do arquiteto
└─ MEMORY.md                           # Memória de contexto
```

3. ONTOLOGIA FRACTAL — ESTRUTURA DE DADOS

3.1 Hierarquia N0→N4

N0 (Vetor)	2 nós	SINTRÓPICO ENTRÓPICO
└─ N1 (Pilar)	6 nós	LOGOS, BIOS, PATHOS, KHAOS, APEIRON, MYTHOS



Cardinalidade: $2 \times 3 \times 3 \times 3 \times 3 = 162$ células N4

3.2 Esquema de um N4 Enriquecido

```

{
  "uid": "N4_DECOMPOSICAO",
  "path": "S1.L1.1.1-A",
  "nome": "Decomposição",
  "nome_normalizado": "decomposicao",
  "descricao": "...",
  "natureza": "processo",
  "funcao_cognitiva": ["decompor", "segmentar", "modularizar"],
  "assinatura_semantica": {
    "abstracao": 0.82,
    "complexidade": 0.71,
    "causalidade": 0.65,
    "emocionalidade": 0.12,
    "materialidade": 0.34,
    "simbolismo": 0.15,
    "dinamismo": 0.78,
    "temporalidade": 0.55,
    "ambiguidade": 0.08
  },
  "relacoes": [
    {
      "tipo": "depende_de",
      "alvo": "N4_ANALISE",
      "peso": 0.82
    },
    {
      "tipo": "complementa",
      "alvo": "N4_INTEGRACAO",
      "peso": 0.71
    }
  ],
  "opostos": ["N4_HOLISMO"],
  "tags": ["analise", "estrutura", "metodo"],
  "exemplos": [...],
  "analogias": [...],
  "heranca": {
    "n0": "SINTRÓPICO",
    "n1": "LOGOS",
    "n2": "ALGORITMIA",
    "n3": "RESOLUÇÃO"
  }
}

```

```
}
}
```

3.3 Tipagem Ontológica

10 naturezas possíveis para cada célula:

Natureza	Uso
processo	Ações, transformações, fluxos
estado	Condições estáveis, snapshots
fenomeno	Observáveis, manifestações
principio	Leis fundamentais, axiomas
mecanismo	Máquinas causais, engrenagens
estrutura	Formas, organização espacial
arquétipo	Padrões primordiais, símbolos
dinamica	Forças, movimentos, tensões
restricao	Limites, proibições, fronteiras
vetor	Direções, tendências, trajetórias

3.4 Tipos de Relação

10 tipos tipados com peso [-1.0, +1.0]:

Tipo	Semântica	Peso típico
depende_de	Pré-requisito	0.8–1.0
complementa	Compleitude	0.6–0.9
contrasta	Oposição	-0.5 a -0.9
expande	Generalização	0.5–0.8
implementa	Concretização	0.7–0.9
generaliza	Abstração	0.6–0.8
especializa	Refinamento	0.6–0.8
causa	Causalidade	0.7–0.95
equilibra	Homeostase	0.5–0.8
transforma	Metamorfose	0.6–0.9

3.5 Assinatura Semântica (9D)

Cada N4 possui um vetor de 9 dimensões contínuas [0.0, 1.0]:

Dimensão	Significado
abstracao	Nível de abstração conceitual
complexidade	Grau de complexidade estrutural
causalidade	Força de relações causais
emocionalidade	Carga emocional
materialidade	Tangibilidade física
simbolismo	Densidade simbólica
dinamismo	Grau de mudança/movimento
temporalidade	Relação com o tempo
ambiguidade	Grau de indeterminação

4. MÓDULOS DO SISTEMA

4.1 Core (framework agnóstico)

Módulo	Arquivo	Responsabilidade
OntologyGraph	core/ontology_graph.py	Grafo NetworkX com 242 nós, métricas topológicas, export GEXF/JSON
SignatureEngine	core/signature_engine.py	Geração e comparação de assinaturas 9D
DialecticEngine	core/dialectic_engine.py	Inferência de opostos, tensões, distância dialética (5 níveis)
CognitiveFunctionEngine	core/cognitive_function_engine.py	Inferência e expansão de funções cognitivas
OntologyTyping	core/ontology_typing.py	Atribuição e validação de naturezas
RelationEngine	core/relation_engine.py	Construção, validação e inferência de relações reversas
BaseAgent	core/agent_base.py	Classe base para agentes com memória e result types

4.2 Runtime (pipeline)

Módulo	Arquivo	Linhas	Responsabilidade
OntologicalClassifier	runtime/classifier.py	949	Classificação multinível N0→N4 com keywords + embeddings
CognitiveRouter	runtime/router.py	450	Roteamento por pilar/estratégia cognitiva
HybridRetriever	runtime/retriever.py	475	Retrieval híbrido: vetorial + gráfico + simbólico
OntologySynthesizer	runtime/synthesizer.py	647	Composição de resposta com templates por pilar
OntologyValidator	runtime/validator.py	732	5 verificações de coerência ontológica
MultiAgentOrchestrator	runtime/orchestrator.py	563	Coordenação do pipeline completo

4.3 Agents (.kilo/agents/)

Agente	Arquivo	Linhas	Responsabilidade
ClassifierAgent	classifier_agent.py	360	Classificação com caching e confiança calibrada
GraphAgent	graph_agent.py	454	Expansão de contexto via BFS ponderado
SynthesisAgent	synthesis_agent.py	292	Montagem de prompts dinâmicos por pilar
ValidatorAgent	validator_agent.py	471	Verificação multi-camada
DialecticAgent	dialectic_agent.py	545	Inferência de opostos multi-camada
MultiAgentOrchestrator	multi_agent_orchestrator.py	~480	Coordenação de todos os agentes
AutonomyLayer	autonomy_layer.py	~550	Auto-avaliação, feedback loops, thresholds adaptativos

5. GRAPH ENGINE — DETALHES

5.1 Topologia Atual

Nós: 242 (2 N0 + 6 N1 + 18 N2 + 54 N3 + 162 N4)

Arestas: ~2.557

- Hierárquicas (pai→filho): ~240 (peso 1.0)
- Relacionais (N4↔N4): ~2.317 (pesos variados)

5.2 Métricas Calculadas

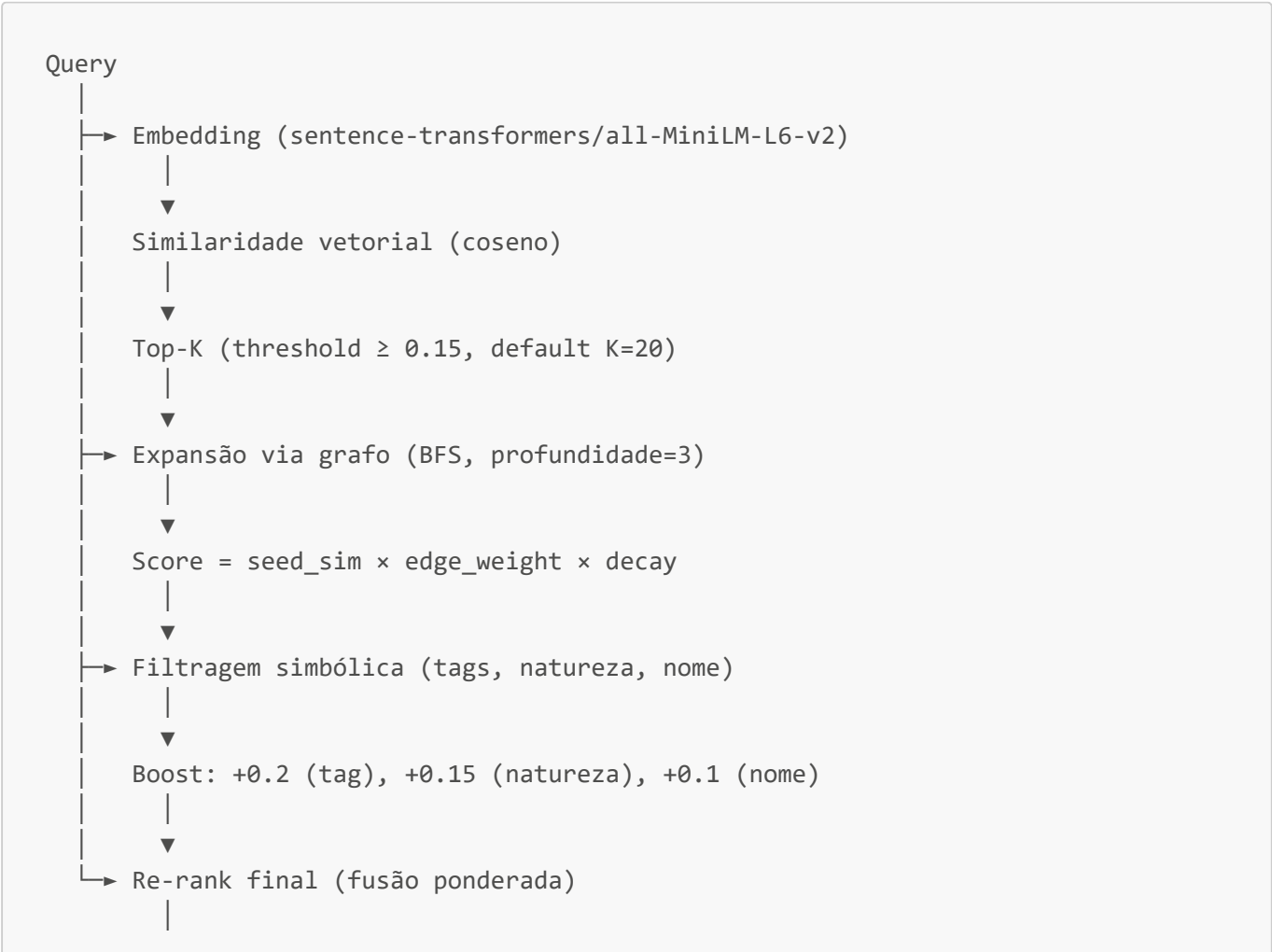
- Centralidade de grau, intermediação e proximidade
- Detecção de comunidades (Louvain)
- Densidade do grafo
- Pontes e nós isolados
- Caminhos semânticos mais curtos

5.3 Exportações

Formato	Arquivo	Tamanho	Uso
GEXF	data/graphs/ontology.gexf	891KB	Visualização (Gephi, NetworkX)
JSON	data/graphs/ontology.json	603KB	Interoperabilidade
Metrics	data/graphs/graph_metrics.json	86KB	Análise topológica

6. RETRIEVAL HÍBRIDO

6.1 Pipeline de Retrieval



▼
Top-K saída (default 12)

6.2 Pesos de Fusão

```
fusao:
  pesos:
    vetorial: 0.50
    grafico: 0.30
    simbolico: 0.20
  top_k_saida: 12
  limiar_final: 0.40
```

6.3 Embeddings

- **Modelo:** `sentence-transformers/all-MiniLM-L6-v2`
- **Dimensão:** 384
- **Quantidade:** 162 vetores (um por N4)
- **Local:** `data/embeddings/N4_*.npy`
- **Formato:** float32, normalizado (coseno)

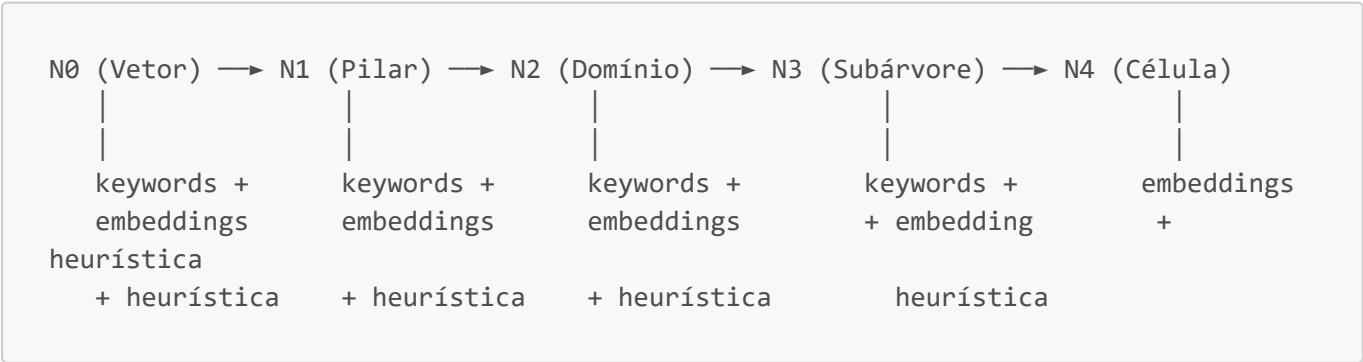
7. CLASSIFICADOR ONTOLÓGICO

7.1 Estratégia Multi-Sinal

O classificador combina 4 sinais em cascata:

1. **Keywords léxicas** (match direto, alta precisão)
2. **Embeddings semânticos** (similaridade vetorial)
3. **Regras simbólicas** (herança hierárquica)
4. **LLM fallback** (quando sinais são ambíguos)

7.2 Cascata de Decisão



7.3 Confiança Calibrada

A confiança final é calculada como combinação ponderada:

- Score léxico: peso alto (0.4) quando há match exato
- Score semântico: peso médio (0.35)
- Score hierárquico: peso médio (0.15)
- Consistência multi-nível: peso (0.10)

8. VALIDADOR ONTOLÓGICO

8.1 Cinco Verificações

#	Verificação	O que testa
1	<code>eixo_coerente</code>	Pilar pertence ao eixo correto
2	<code>assinatura_preservada</code>	Dimensões semânticas dentro dos limites esperados por pilar
3	<code>polaridades_compativeis</code>	Sem mistura de opostos (LOGOS↔KHAOS, etc.)
4	<code>conceitos_validos</code>	Referências N4 existem no registry
5	<code>relacoes_consistentes</code>	Sem contradições no grafo

8.2 Limites de Assinatura por Pilar

Cada pilar tem faixas min/max esperadas para as 9 dimensões (ver seção 4.2, `runtime/validator.py` linhas 43-110).

9. ENTRY POINTS

9.1 CLI Principal (`run_kilo.py`)

```
# Health check (13 verificações)
python run_kilo.py --health

# Query simples
python run_kilo.py --query "como resolver conflitos?"

# Pipeline completo
python run_kilo.py --query "como otimizar um processo de produção?" --mode full

# Modo interativo
python run_kilo.py --mode interactive

# Modo autônomo (com auto-avaliação)
python run_kilo.py --query "..." --mode autonomous

# Debug
python run_kilo.py --query "..." --mode full --verbose --save-trace
```

9.2 Modos de Pipeline

Modo	Etapas	Uso
full	classifier → router → retriever → graph → dialectic → synthesis → validator	Queries complexas
simple	router → retriever → synthesis → validator	Queries simples, baixa latência
autonomous	full + autonomy layer	Auto-avaliação e feedback

10. CONFIGURAÇÕES

10.1 Thresholds Críticos

```
retrieval:
  semantic_threshold: 0.72
  graph_depth: 3
  rerank_limit: 12

signatures:
  abstraction_weight: 1.2
  causality_weight: 1.1
  symbolism_weight: 0.9

relations:
  complementa: 0.8
  depende_de: 1.0
  contrasta: -0.7
```

10.2 Config Files

Arquivo	Propósito
.kilo/config/ontology.yaml	Parâmetros ontológicos
.kilo/config/embedding.yaml	Modelo e dimensões
.kilo/config/retrieval.yaml	Pesos e thresholds
.kilo/config/graph.yaml	Parâmetros do grafo

11. MAPA DE ESTADO ATUAL vs. ROADMAP

Já Implementado e Validado 

Item	Status	Evidência
162 JSONs N4 enriquecidos		data/json/ — 162 arquivos com uid, natureza, relações, assinaturas

Item	Status	Evidência
UIDs permanentes	✓	<code>tools/uid_generator.py</code> + UIDs em todos os JSONs
Naturezas ontológicas	✓	10 tipos atribuídos, validados por <code>core/ontology_typing.py</code>
Relações tipadas	✓	10 tipos com pesos em <code>core/relation_engine.py</code>
Assinaturas semânticas 9D	✓	<code>core/signature_engine.py</code> + dados em cada JSON
Opostos ontológicos	✓	<code>core/dialectic_engine.py</code> (1018 linhas, 5 níveis)
Funções cognitivas	✓	<code>core/cognitive_function_engine.py</code>
Grafo semântico	✓	242 nós, ~2557 arestas, <code>data/graphs/ontology.gexf</code>
Métricas de grafo	✓	<code>data/graphs/graph_metrics.json</code> (86KB)
Embeddings 384-dim	✓	162 arquivos <code>.npy</code> em <code>data/embeddings/</code>
Retrieval híbrido	✓	<code>runtime/retriever.py</code> — vetorial + gráfico + simbólico
Classificador N0→N4	✓	<code>runtime/classifier.py</code> (949 linhas)
Router cognitivo	✓	<code>runtime/router.py</code> (450 linhas)
Síntese ontológica	✓	<code>runtime/synthesizer.py</code> (647 linhas)
Validador	✓	<code>runtime/validator.py</code> (732 linhas)
Orquestrador multi-agente	✓	<code>runtime/orchestrator.py</code> + <code>.kilo/agents/</code>
6 agentes especializados	✓	classifier, graph, synthesis, validator, dialectic, orchestrator
Autonomy Layer	✓	<code>.kilo/agents/autonomy_layer.py</code> (~550 linhas)
Health Check	✓	13/13 verificações passam

Item	Status	Evidência
Pipeline full + simple	✓	Testados e operacionais
Prompt templates	✓	<code>.kilo/prompts/{classifier,retrieval,routing,synthesis,validation}/</code>
Entry point CLI	✓	<code>run_kilo.py</code> com modos health/query/interactive/full/autonomous

Pendente 

Item	Prioridade	Esforço Estimado
DOCS/USAGE.md — documentação completa de APIs e uso	P0	2–3 horas
docs/arquitetura_sistema_cognitivo.md — este documento	P0	✓ feito
Corrigir warnings do validator (5 self-loops em <code>é_requisito_de</code>)	P1	1–2 horas
Corrigir 5 warnings de assinatura fora dos limites	P1	1–2 horas
Cache persistente de embeddings (warm-up lento no 1º carregamento)	P1	2–3 horas
Consolidar <code>driver.py</code> → usar <code>run_kilo.py</code> como entry point único	P1	1 hora
Validar/expandir suite de testes (<code>tests/</code>)	P2	4–6 horas
Implementar episodic + semantic memory storage	P2	6–8 horas
Refinar templates YAML por agente	P2	2–3 horas
Métricas de runtime + logging estruturado + health endpoint HTTP	P3	4–6 horas
Preparar migração para Neo4j/ArangoDB	P3	4–8 horas
MCP server para integração externa	P3	6–10 horas
Multi-tenant support	P3	8–12 horas

12. DECISÕES ARQUITETURAIS

12.1 Por que NetworkX em vez de Neo4j?

Decisão: NetworkX para a fase atual. Neo4j planejado para escala.

Justificativa:

- 242 nós e ~2.5K arestas cabem perfeitamente em memória
- NetworkX oferece API rica para métricas topológicas sem infraestrutura externa
- A arquitetura do `OntologyGraph` abstrai o backend — a migração para Neo4j requer apenas reimplementar o driver interno

- Zero dependências operacionais para deploy inicial

12.2 Por que sentence-transformers em vez de TF-IDF?

Decisão: sentence-transformers como default, TF-IDF como fallback.

Justificativa:

- TF-IDF falha em capturar semântica latente (ex: "resolver" ≈ "solucionar")
- `all-MiniLM-L6-v2` é leve (80MB), rápido (~10ms/query) e eficaz para português
- Fallback TF-IDF garante funcionamento sem GPU/sem modelo externo
- Hybrid retrieval combina ambos via re-rank

12.3 Por que Dataclasses em vez de ORM?

Decisão: Dataclasses puras para resultados e configuração.

Justificativa:

- Sem estado mutável compartilhado — cada pipeline roda como instância isolada
- Serialização/deserialização simples via `to_dict()`
- Zero dependências de banco de dados para o core
- Memória eficiente para 162 entidades

12.4 Por que múltiplos engines em vez de um monolítico?

Decisão: 7 engines especializados + 6 agents + 1 orchestrator.

Justificativa:

- Cada engine tem um contrato claro (input → output)
- Testabilidade isolada
- Substituição independente (ex: trocar retriever sem afetar classifier)
- Degradação graciosa (falha de um módulo não derruba o sistema)
- Alinhamento com o DNA ontológico: cada módulo reflete um princípio (LOGOS = estrutura, KHAOS = adaptabilidade)

13. PADRÕES DE PROJETO APLICADOS

Padrão	Onde	Por quê
Strategy	<code>router.py</code> — estratégias por pilar	Trocar algoritmo de routing sem alterar orchestrator
Chain of Responsibility	Pipeline — cada etapa recebe e enriquece contexto	Extensível, cada agente é um elo
Template Method	<code>synthesizer.py</code> — templates por pilar	Comportamento varia por pilar, estrutura é fixa

Padrão	Onde	Por quê
Observer	<code>autonomy_layer.py</code> — feedback loops	Monitoramento reativo sem acoplamento
Factory	<code>classifier.py</code> — construção de features	Diferentes sinais (léxico, semântico, simbólico)
Facade	<code>orchestrator.py</code> — interface única	Complexidade oculta, API simples
Flyweight	<code>ontology_graph.py</code> — registry compartilhado	162 células reutilizadas em múltiplos contextos

14. LIMITAÇÕES CONHECIDAS

1. **Carregamento de embeddings:** Primeira query leva ~10s para carregar modelo sentence-transformers. Mitigação: cache em disco ou warm-up no startup.
2. **Windows compatibility:** `driver.py` tem path issues no Windows. `run_kilo.py` é o entry point recomendado.
3. **Validator warnings:** 5 self-loops em relações `é_requisito_de` e 5 assinaturas ligeiramente fora dos limites esperados. Não bloqueiam, mas merecem investigação.
4. **Sem persistência de memória:** O AutonomyLayer mantém estado em memória volátil. Reinícios perdem episódios.
5. **Sem interface HTTP:** Acesso via CLI apenas. Para integração web, seria necessário um wrapper Flask/FastAPI.
6. **Sem versionamento de ontologia:** Alterações nos JSONs não são versionadas. Git é o único controle.

15. EXTENSÕES FUTURAS (ROADMAP V2)

Fase N5 — Micro-Regiões Cognitivas

- Subdivisão de cada N4 em microestados N5
- Cada N5 com assinatura vetorial própria
- 486 células (162 × 3)

Fase N6 — Semantic Anti-Collider

- Detecção de colisões semânticas entre células
- Resolução automática de ambiguidade
- Controle de entropia ontológica

Fase N7 — Cognitive Expression Layer

- Adaptação da resposta ao perfil cognitivo do usuário
- Compressão semântica adaptativa

- Modulação expressiva dinâmica

Fase N8 — Semantic Memory System

- Memória episódica (histórico de interações)
- Memória semântica (conhecimento persistente)
- Memória de atratores (padrões recorrentes)
- Memória de trajetória (evolução temporal)

Fase N9 — Infraestrutura

- Migração para Neo4j/ArangoDB
- MCP server para integração externa
- API REST/FastAPI
- Multi-tenant
- Observabilidade (metrics, tracing, logging)

16. MÉTRICAS DE SAÚDE

Métrica	Valor Atual	Status
Células N4 carregadas	162/162	✓
Embeddings carregados	162/162	✓
Nós no grafo	242	✓
Arestas no grafo	~2.557	✓
Health checks passados	13/13	✓
Pipelines testados	full + simple	✓
Agentes operacionais	6/6 + orchestrator	✓
Autonomy Layer	Inicializado	✓
Documentação	Parcial	●
Testes automatizados	Inexistentes	●
Persistência de memória	Não implementada	●
Interface HTTP	Não implementada	●